

## Лекция 6. Контроль версий конфигураций и сценариев: Git

Git используется администратором для фиксации изменений в конфигурациях, сценариях и документации

Главная идея лекции: изменение должно быть проверяемым, объяснимым и обратимым

Учебный сценарий: репозиторий linux-infrastructure для Bash-сценариев и безопасных копий конфигураций

Работа ведется с текстовыми файлами, без хранения паролей, ключей и live-конфигураций из /etc напрямую

## Цель лекции

Понять назначение Git в работе системного администратора

Уметь объяснить Working Tree, Staging Area, Commit, Branch и Remote Repository

Уметь выполнить безопасный цикл изменения: проверить, добавить, зафиксировать, сравнить и восстановить

Уметь объяснить ограничения Git при хранении системных конфигураций и секретов

## Учебные вопросы

1. Контроль версий для системного администратора
2. Working Tree, Staging Area и локальный репозиторий
3. Репозиторий, commits, status, log и diff
4. Сообщения commit, восстановление и базовое ветвление
5. .gitignore, конфигурации, документация и Bash-сценарии
6. Запрет хранения секретов
7. Отличие Git от backup и структура инфраструктурного репозитория

## Учебный сценарий лекции

Администратор ведет каталог linux-infrastructure

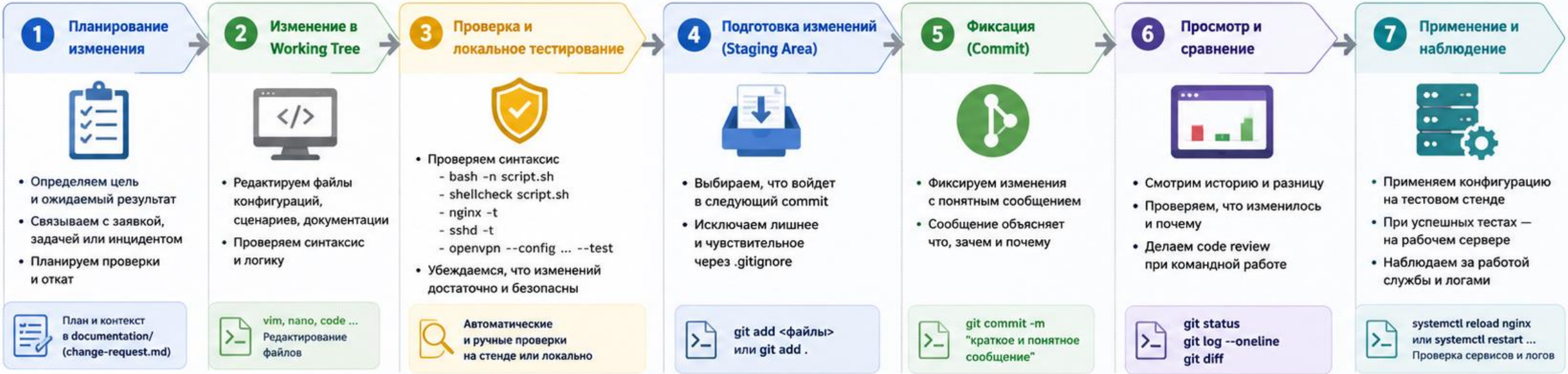
В каталоге хранятся scripts, configs, documentation, tests и README.md

Перед commit выполняются проверки: bash -n, shellcheck, nginx -t, sshd -t или команда конкретной службы

После commit история должна объяснять, кто, что и зачем изменил

# Схема: Git в жизненном цикле изменения конфигурации

Git помогает администратору безопасно вносить изменения, проверять их, документировать и при необходимости откатывать.



### Результат

- ✓ Изменение проверяемое: есть проверки и тесты
- ✓ Изменение объяснимое: есть commit и причина
- ✓ Изменение обратимое: можно вернуться к рабочему состоянию
- ✓ История хранится в безопасном удаленном репозитории

**ВАЖНО**

Никогда не храните пароли, ключи и live-конфигурации из /etc!  
Используйте примеры и шаблоны, .gitignore и безопасные практики.

**Типичная структура репозитория linux-infrastructure**

```
linux-infrastructure/  
├── scripts/  
├── configs/  
├── documentation/  
├── tests/  
├── .gitignore  
└── README.md
```

# Bash-сценарии и утилиты  
# Конфигурации без секретов  
# Документация (MD)  
# Тесты и проверки

**Примеры предварительных проверок**

`bash -n script.sh`

`shellcheck script.sh`

`nginx -t`

`sshd -t`

`openvpn --config ... --test`

**Откат (если что-то пошло не так)**

Найти стабильный commit в git log → `git checkout <commit>` или `git restore <файл>` → Применить конфигурацию и перезапустить службу → Проверить работу службы

- Планирование
- Редактирование
- Проверка
- Подготовка
- Commit
- Проверка изменений
- Применение

# 1. Контроль версий для системного администратора

Контроль версий — это способ хранить историю изменений файлов и возвращаться к понятным состояниям

Для администратора Git полезен не как инструмент программирования, а как журнал технических решений

Каждое изменение конфигурации должно иметь причину, автора, дату и проверяемый результат

Без истории трудно понять, почему служба работала вчера и перестала работать сегодня

## Git полезен в эксплуатационной работе

Bash-сценарии меняются по мере появления новых проверок и автоматизации

Конфигурации Nginx, SSH, OpenVPN и systemd требуют аккуратного сравнения версий

Документация по адресации, топологии и процедурам восстановления должна меняться вместе с конфигурациями

Commit помогает связать техническое изменение с причиной: заявкой, ошибкой, лабораторным заданием или инцидентом

## Что имеет смысл хранить в Git

Текстовые конфигурации без секретов: шаблоны Nginx, systemd unit, SSH client config

Bash-сценарии и вспомогательные проверки

Markdown-документация: topology.md, addressing.md, recovery.md

Примеры параметров: .env.example, secrets.conf.example, client-credentials.example

Тестовые сценарии, которые проверяют синтаксис и ожидаемое состояние

## Что Git не решает автоматически

Git показывает, что изменилось, но не доказывает, что служба работает

Git не применяет конфигурацию к Nginx, SSH, OpenVPN или systemd

Git не заменяет тестовый стенд и процедуру проверки перед внедрением

Git не защищает секрет, если пользователь уже добавил его в историю

Git не хранит полноценное состояние сервера: процессы, базы, журналы и данные пользователей

## Git не является backup и config management

Backup восстанавливает данные и состояние после потери или аварии

Git хранит историю текстовых файлов и помогает понять изменения

Config management применяет состояние на серверы автоматически: например Ansible

Remote Repository повышает сохранность истории, но не является полноценной резервной копией сервера

В реальной эксплуатации Git, backup и автоматизация дополняют друг друга

## 2. Working Tree, Staging Area и локальный репозиторий

Working Tree — рабочий каталог, где администратор редактирует файлы

Staging Area или Index — область подготовки будущего commit

Local Repository — каталог .git с историей проекта на локальной машине

Commit сохраняет не все файлы из Working Tree, а только подготовленное состояние из Staging Area

## Три области Git нужно различать строго

Working Tree содержит текущие файлы проекта

Staging Area отвечает на вопрос: что войдет в следующий commit

Repository хранит уже зафиксированные состояния

Один и тот же файл может быть изменен в Working Tree, но еще не подготовлен к commit

Ошибка новичка: думать, что git commit автоматически сохраняет все изменения

## Состояния одного файла

Untracked — файл есть в каталоге, но Git еще не отслеживает его

Staged — файл подготовлен командой `git add` и войдет в следующий `commit`

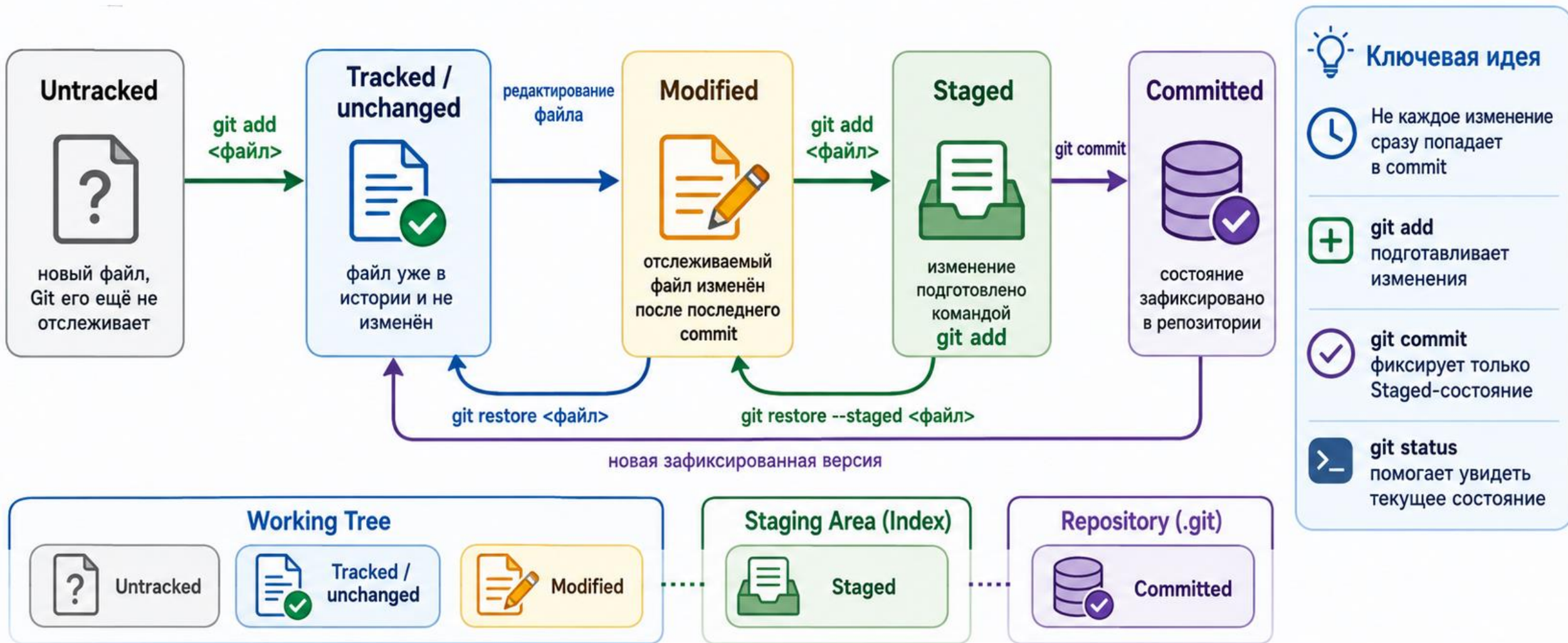
Tracked и unchanged — файл уже в истории и не изменен относительно последнего `commit`

Modified — файл отслеживается, но его содержимое изменилось после последнего `commit`

Понимание состояний помогает не потерять нужные изменения и не добавить лишние файлы

# Схема: состояния файла в Git

Как файл перемещается между состояниями в Working Tree, Staging Area и Repository



## Важно

Частая ошибка новичка: думать, что `git commit` сохраняет все изменения автоматически. На самом деле `commit` берёт только то, что находится в Staging Area.

## Первичная настройка Git

Git должен знать имя и адрес автора commit

Команда: `git config --global user.name 'Student Name'`

Пример: имя автора будет записано в каждый новый commit этого пользователя

Команда: `git config --global user.email 'student@example.test'`

Проверка: `git config --global --list`

## Основную ветку лучше задать сразу

В разных версиях Git начальная ветка может называться `main` или `master`

Команда: `git init -b main`

Пример: репозиторий создается сразу с основной веткой `main`

Если `git init -b main` не поддерживается: `git init`, затем `git branch -M main`

Иначе команда `git switch main` может не сработать в лаборатории

## **git status показывает текущее состояние**

Команда: `git status`

Назначение: показать текущую ветку, измененные файлы и Staging Area

Пример: после создания README.md команда покажет файл как Untracked

Краткий режим: `git status --short`

Привычка администратора: начинать и заканчивать работу с `git status`

## **git add подготавливает только выбранные изменения**

Команда: `git add scripts/server-health.sh`

Назначение: добавить конкретный файл в Staging Area

Пример: после `git add` файл готов к `commit`, но история еще не создана

Механическое `git add .` опасно: можно случайно добавить ключи, PCAP, дампы или временные файлы

Перед `git add` нужно выполнить `git status` и `git diff`

## **git add -p помогает разделять изменения**

Команда: `git add -p`

Назначение: интерактивно добавить отдельные фрагменты файла

Пример: в одном Bash-сценарии можно добавить проверку диска, но не добавлять экспериментальную проверку сети

Это помогает делать маленькие и понятные commits

Для администратора это особенно важно при одновременной правке конфигурации и комментариев

### 3. Репозиторий, commits, status, log и diff

Репозиторий — это проект с историей изменений

Commit — зафиксированное состояние  
подготовленных файлов

Log показывает последовательность commits

Diff показывает различия между состояниями

Show позволяет посмотреть конкретный commit

## Базовая структура инфраструктурного репозитория

`configs` — безопасные копии и шаблоны конфигураций

`scripts` — Bash-сценарии диагностики, проверки и обслуживания

`documentation` — топология, адресация, процедуры восстановления

`examples` — файлы-примеры вместо настоящих секретов

`tests` — сценарии проверки синтаксиса и ожидаемого состояния

## Первый рабочий цикл команд

Команда: `git status` — проверить, что изменилось

Команда: `git diff` — увидеть содержимое изменений до подготовки

Команда: `git add scripts/server-health.sh` — подготовить нужный файл

Команда: `git diff --staged` — проверить будущий commit

Команда: `git commit -m 'Add server health check'` — зафиксировать изменение

## Разные варианты git diff

git diff сравнивает Working Tree и Staging Area

git diff --staged сравнивает Staging Area и последний commit

git diff HEAD показывает все текущие изменения относительно HEAD

git show HEAD показывает последний завершённый commit

git diff HEAD~1..HEAD показывает различия только между двумя commit

## Почему `git diff HEAD~1` может запутать

Команда `git diff HEAD~1` сравнивает Working Tree с предыдущим commit

Если есть незакоммиченные изменения, они тоже попадут в вывод

Для просмотра последнего завершенного изменения лучше использовать `git show HEAD`

Альтернатива: `git diff HEAD~1..HEAD`

В учебной работе важно отличать историю от текущих незавершенных правок

## **git log и git show помогают читать историю**

Команда: `git log --oneline --graph --decorate -10`

Назначение: показать последние commits, ветки и форму истории

Команда: `git show HEAD`

Назначение: показать сообщение, автора и изменения последнего commit

В расследовании аварии история Git часто отвечает на вопрос: какое изменение было последним

## Сообщение commit должно объяснять изменение

Плохой пример: update или fix

Хороший пример: Add route for OpenVPN client network

Первая строка должна кратко описывать действие

При необходимости тело commit объясняет причину, ограничения и способ проверки

Сообщение commit пишется для будущего администратора, который будет искать причину проблемы

## Перед commit нужна техническая проверка

Git доказывает, что файл изменился, но не доказывает, что конфигурация работает

Для Bash: `bash -n scripts/server-health.sh`

Для Bash дополнительно: `shellcheck scripts/server-health.sh`

Для Nginx: `sudo nginx -t`

Для SSH: `sudo sshd -t`

## Git хранит исполняемый признак файла ограниченно

Для Bash-сценария важно, чтобы файл можно было запускать

Команда: `chmod +x scripts/server-health.sh`

Команда Git: `git update-index --chmod=+x scripts/server-health.sh`

Git обычно хранит executable bit, но не все права, владельцев и ACL

Права применения на сервере нужно описывать отдельно в README или deployment script

## 4. Сообщения commit, восстановление и базовое ветвление

Commit должен быть небольшим и логически завершенным

Восстановление файла зависит от того, где находится изменение: Working Tree, Staging Area или Repository

Ветка помогает изолировать изменение в репозитории

Merge объединяет историю, но после него все равно нужна проверка результата

Конфликт решается не только удалением маркеров, а проверкой итоговой конфигурации

## Восстановление нужно разделять по сценариям

Сценарий 1: отменить незакоммиченные изменения в Working Tree

Сценарий 2: убрать файл из Staging Area, сохранив правки

Сценарий 3: взять старую версию файла из другого commit

Сценарий 4: отменить уже опубликованное изменение новым commit

Нельзя одной командой объяснять все виды восстановления

## Отмена изменений в Working Tree

Команда: `git restore configs/nginx/app.conf`

Назначение: отменить незакоммиченные изменения файла

Пример: администратор случайно испортил шаблон Nginx и возвращает его к последнему подготовленному или зафиксированному состоянию

Перед этим обязательно выполнить: `git status`

Перед этим обязательно выполнить: `git diff -- configs/nginx/app.conf`

## Убрать файл из Staging Area

Команда: `git restore --staged configs/nginx/app.conf`

Назначение: убрать файл из будущего commit, но оставить изменения в Working Tree

Пример: конфигурация Nginx была добавлена слишком рано, а проверка `nginx -t` еще не выполнена

После команды файл остается измененным, но больше не находится в Staging Area

Проверка: `git status`

## Восстановить версию из другого commit

Команда: `git restore --source=HEAD~1  
configs/nginx/app.conf`

Назначение: взять версию файла из предыдущего commit в Working Tree

После восстановления нужно выполнить проверку службы: `sudo nginx -t`

Затем создать новый commit: `git add  
configs/nginx/app.conf`

Пример commit: `git commit -m 'Restore previous Nginx configuration'`

## **git restore может уничтожить полезные правки**

git restore потенциально разрушителен для  
незакоммиченных изменений

Git не восстановит содержимое, которое никогда не  
было добавлено в Staging Area или Repository

Перед восстановлением полезно сохранить копию: `cp  
-a configs/nginx/app.conf /tmp/app.conf.before-restore`

Альтернатива для опытного пользователя: временный  
`commit` или `git stash`

В базовой работе главное правило: сначала `status` и  
`diff`, потом `restore`

## Ветка изолирует изменение в репозитории

Branch — отдельная линия разработки изменений

Команда: `git switch -c test-openvpn-route`

Назначение: создать ветку и сразу перейти в нее

Пример: маршрут OpenVPN проверяется отдельно от основной ветки `main`

Ветка полезна для эксперимента, но не заменяет тестовую виртуальную машину

## Переключение ветки меняет Working Tree

Команда `git switch` меняет файлы в рабочем каталоге  
Если репозиторий связан с live-файлами службы,  
переключение ветки может изменить действующую  
конфигурацию

Поэтому репозиторий с шаблонами и очищенными  
копиями лучше держать отдельно от `/etc`

Безопасная схема: Git repository, validation, test  
environment, approved deployment, live configuration

На сервере не следует экспериментировать ветками  
прямо внутри `/etc`

## Merge объединяет ветку с основной линией

Перед merge нужно перейти в основную ветку: `git switch main`

Команда: `git merge test-openvpn-route`

Fast-Forward просто передвигает указатель ветки вперед

Merge Commit создается, когда истории веток разошлись

После merge обычный `git diff` может ничего не показать, потому что изменения уже вошли в историю

# Диаграмма: ветка, merge и конфликт

Ветки позволяют работать независимо. Merge объединяет изменения.

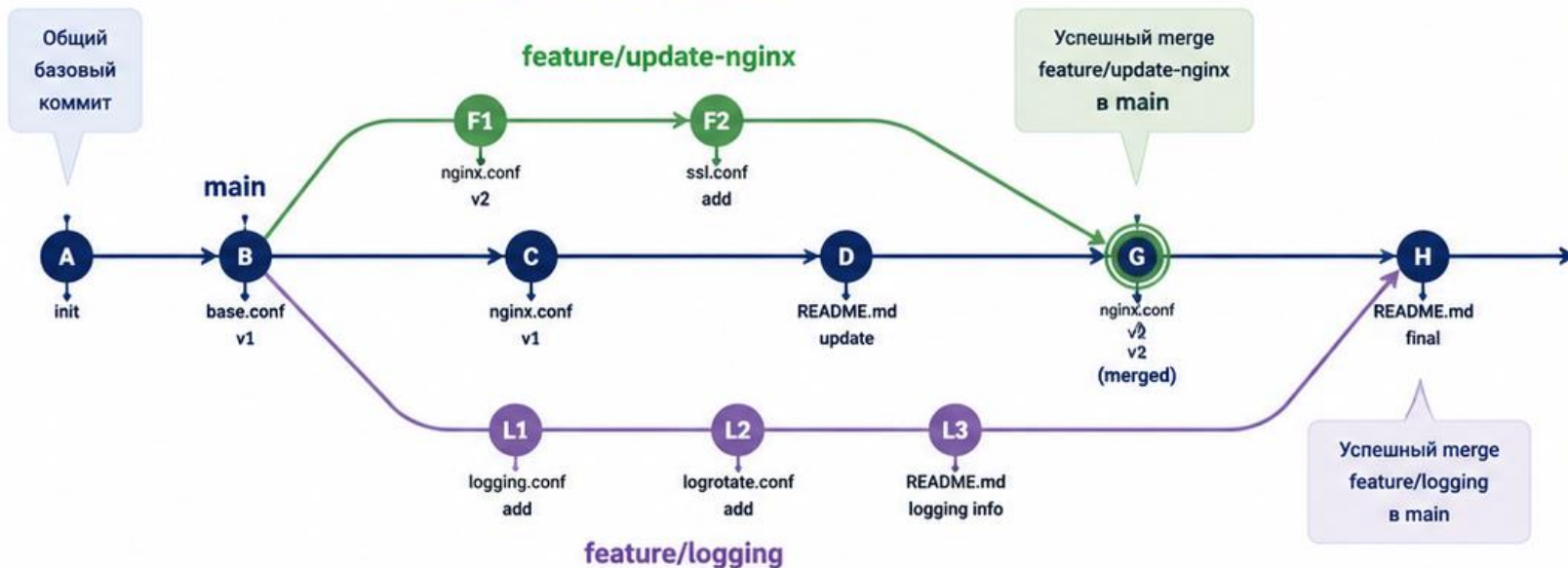
Если изменения затрагивают одни и те же строки — возникает конфликт.

## Условные обозначения

- main (production)
- feature/update-nginx
- feature/logging

- ✓ Успешный merge
- ⚠ Конфликт

## Как развиваются ветки



## Конфликт при merge

Обе ветки изменили одни и те же строки в nginx.conf

main (D)

```
1 server {
2   listen 80;
3   server_name site.local;
4 }
```

feature/update-nginx (F1)

```
1 server {
2   listen 8080; # изменено
3   server_name site.local;
4 }
```

Конфликт

```
$ git merge feature/update-nginx
Auto-merging nginx.conf
CONFLICT (content): Merge conflict in nginx.conf
Automatic merge failed; fix conflicts and then commit the result.
```

## Разрешение конфликта

- 1 Открой файл
  - 2 Добавь файл
  - 3 Закоммить merge
- Исправь конфликт вручную
- git add nginx.conf
- git commit (сообщение о merge)

• Итоговый файл после разрешения

```
1 server {
2   listen 8080;
3   server_name site.local;
4 }
```

✓ Конфликт разрешён.  
История сохранена.

## Что даёт работа с ветками

- Изоляция изменений**  
Работайте над задачей, не влияя на main
- Безопасные эксперименты**  
Можно пробовать идеи и легко откатиться
- Понятная история**  
Каждый шаг описан в коммитах
- Совместная работа**  
Легко делиться, проверять и обсуждать
- Надёжность**  
Merge и история предотвращают потери

## Полезные команды

```
$ git branch # список веток
$ git branch <name> # создать ветку
$ git checkout <branch> # переключиться на ветку
$ git checkout -b <name> # создать и переключиться
$ git merge <branch> # объединить ветку в текущую
$ git log --graph --oneline --all # граф истории
```

## Стандартный поток работы



## Рекомендации

- ✓ Делайте маленькие и понятные commit
- ✓ Пишите осмысленные сообщения commit
- ✓ Проверяйте код перед merge (тесты, линтеры)
- ✓ Не храните секреты и ключи в репозитории
- ✓ Регулярно синхронизируйте ветки с main

## Главное

Ветки помогают работать независимо. Merge объединяет изменения. Конфликты — нормальная часть совместной работы.

## Конфликт требует осознанного решения

Маркеры конфликта: <<<<<< HEAD, =====, >>>>>>

test-openvpn-route

HEAD — текущая версия в ветке, куда выполняется  
merge

Вторая часть — версия из объединяемой ветки

После ручного исправления: git add  
configs/openvpn/server.conf

Если merge выполнять не следует: git merge --abort

## После merge проверяют результат и историю

Команда: `git status`

Команда: `git log --oneline --graph --decorate -10`

Команда: `git show HEAD`

Для OpenVPN пример проверки зависит от режима и файла конфигурации

Главное правило: конфликт считается решенным после проверки синтаксиса и работоспособности итоговой конфигурации

## 5. .gitignore, конфигурации, документация и Bash-сценарии

.gitignore описывает файлы, которые Git должен не предлагать к добавлению

Он защищает от случайного добавления временных файлов

Он не является механизмом безопасности

Он не действует на файлы, которые уже отслеживаются Git

Для инфраструктурного репозитория .gitignore должен быть написан до первого commit

## Пример .gitignore для администратора

Исключить временные файлы редакторов: \*~, \*.swp

Исключить локальные копии и дампы: \*.bak, \*.dump

Исключить захваты трафика: \*.pcap, \*.pcapng

Исключить реальные секреты: .env, secrets/, \*.key

Хранить примеры вместо секретов: .env.example и  
secrets.conf.example

## **.gitignore не удаляет уже отслеживаемый файл**

Если файл уже добавлен в commit, добавление шаблона в .gitignore не остановит его отслеживание

Команда: `git rm --cached secrets/client.key`

Назначение: убрать файл из индекса Git, но оставить его в рабочем каталоге

Для каталога: `git rm -r --cached secrets/`

Важно: если это был реальный секрет, он все равно остается в старой истории

## Конфигурации нельзя бездумно копировать из /etc

Live-файл может содержать пароль, токен, приватный ключ или внутреннюю адресацию

В репозитории лучше хранить очищенную копию или шаблон

Путь назначения нужно описать в README, например:  
`/etc/nginx/sites-available/app.conf`

Перед применением нужна проверка синтаксиса и резервная копия live-файла

Git repository должен быть отделен от действующей конфигурации службы

## Git не сохраняет все метаданные системного файла

Git обычно хранит содержимое, путь, символическую ссылку и executable bit

Git обычно не хранит владельца, группу, полный набор Unix permissions, ACL и extended attributes

Git не хранит SELinux context, capabilities и состояние службы

После копирования sshd\_config обратно нужны chown, chmod и при необходимости restorecon

Поэтому инструкции применения должны быть частью документации проекта

## **README должен описывать применение конфигурации**

Для каждого файла указать источник и путь назначения

Указать владельца и права: например root:root и 600 для sshd\_config

Указать команду проверки: sudo sshd -t или sudo nginx -t

Указать команду применения: reload или restart конкретной службы

Указать способ rollback при неудачной проверке

## Структура репозитория должна помогать эксплуатации

`configs/nginx`, `configs/openvpn`, `configs/systemd` —  
очищенные конфигурации и шаблоны

`scripts/server-health.sh` и `scripts/backup-configs.sh` —  
проверяемые Bash-сценарии

`documentation/topology.md`, `addressing.md`,  
`recovery.md` — актуальная документация

`examples/env.example` — пример параметров без  
реальных секретов

`tests/verify-configs.sh` — проверки перед `commit` или  
перед применением

## 6. Запрет хранения секретов

Секрет — это данные, которые дают доступ или раскрывают защищенную инфраструктуру

К секретам относятся пароли, токены, приватные ключи, PSK, SNMP Community и Cookie

Секретом может быть не только пароль, но и VPN-профиль с вложенным ключом

После попадания в Git секрет нужно считать скомпрометированным

Удаление строки новым commit не удаляет секрет из предыдущей истории

## **Копии конфигураций могут раскрывать скрытые секреты**

Опасные фрагменты: password=, token=, private-key=, auth-user-pass

Опасные фрагменты: psk=, community=, bindpw=

Конфигурация может раскрыть внутренние IP-адреса, hostnames и структуру сети

Даже очищенный инфраструктурный репозиторий может быть конфиденциальным

Публичным его можно делать только после осознанного review

## Поиск секретов перед commit

Простой поиск password недостаточен

Команда: `grep -RInE`

`'password|passwd|token|secret|api[_-]?key|private[_-]?key'` .

Назначение: найти типовые имена секретных параметров

Команда: `grep -RIn 'BEGIN .*PRIVATE KEY'` .

Назначение: найти приватные ключи в PEM-формате

## **.gitignore не является защитой секретов**

Файл из .gitignore можно принудительно добавить командой `git add -f`

.gitignore защищает от случайности, но не от ошибочного или намеренного действия

Секреты должны физически храниться вне репозитория

В проекте используют placeholders и .example-файлы

Защиту усиливают review, права доступа, secret scan и серверные проверки

## Порядок действий после утечки секрета

Сначала отозвать или заменить секрет: пароль, токен, ключ или PSK

Определить, был ли репозиторий отправлен на Remote Repository

Ограничить доступ и проверить журналы использования учетных данных

Удалить секрет из текущих файлов и при необходимости очистить историю

Даже после очистки истории старый секрет нельзя считать безопасным

## Файлы-примеры заменяют реальные секреты

Вместо `.env` хранится `.env.example`

Вместо `client-credentials` хранится `client-credentials.example`

Пример содержимого: `VPN_USERNAME=CHANGE_ME` и `VPN_PASSWORD=CHANGE_ME`

В `README` указывается: `cp .env.example .env`

Для локального файла с секретом указывается: `chmod 600 .env`

## Доступ к инфраструктурному репозиторию нужно ограничивать

Нужно определить, кто может читать репозиторий

Нужно определить, кто может выполнять push в main

Для важных изменений требуется review или  
утверждение преподавателя

При увольнении сотрудника или завершении работы  
доступ должен отзываться

Локальный pre-commit hook полезен, но его можно  
обойти; серверный контроль надежнее

## 7. Отличие Git от backup и структура инфраструктурного репозитория

Git хранит историю текстовых артефактов  
инфраструктуры

Backup хранит данные для восстановления после  
аварии

Remote Repository помогает совместной работе, но не  
заменяет backup

Инфраструктурный репозиторий должен быть  
конфиденциальным техническим документом

PCAP, дампы, ISO, журналы и виртуальные диски  
обычно не хранят в Git

Хорошая структура репозитория ускоряет диагностику.

## Remote Repository: первый push

Команда: `git remote add origin`

`ssh://git@git.example.test/linux-infrastructure.git`

Назначение: связать локальный репозиторий с удаленным

Проверка адреса: `git remote -v`

Первый push: `git push -u origin main`

После -u последующие отправки можно выполнять командой `git push`

## **git fetch и git pull нужно различать**

git fetch получает данные с Remote Repository, но не меняет текущую ветку автоматически

git pull выполняет fetch и затем интегрирует изменения через merge или rebase

Для новичка безопаснее: git fetch, затем просмотр истории, затем merge

Команда просмотра: git log --oneline --all --graph

Перед pull Working Tree должен быть чистым или осознанно подготовленным

## Аннотированный tag отмечает принятую версию

Легкий tag допустим для быстрой отметки, но проверенную версию лучше подписывать описанием

Команда: `git tag -a lab6-ok -m 'Lab 6 accepted'`

Проверка: `git show lab6-ok`

Отправка конкретного tag: `git push origin lab6-ok`

Обычный `git push` не всегда отправляет все локальные tags

## Итоги лекции

Git помогает администратору фиксировать и  
объяснять изменения в инфраструктуре

Commit сохраняет подготовленное состояние из  
Staging Area, а не все файлы автоматически

Перед commit нужны status, diff, проверка синтаксиса  
и осознанный выбор файлов

Восстановление, merge и pull требуют понимания  
последствий для Working Tree

Секреты, live-конфигурации, метаданные и backup  
имеют отдельные правила обработки

## Контрольные вопросы

Чем Working Tree отличается от Staging Area и Repository?

Почему `git add .` опасен в инфраструктурном репозитории?

Чем отличаются `git diff`, `git diff --staged` и `git show HEAD`?

Почему `.gitignore` не защищает секреты?

Какие действия нужно выполнить после попадания секрета в Git?

Почему Remote Repository нельзя считать полноценным backup сервера?